ALICIA VÁZQUEZ

APIs Escalables y Disponibles

Alicia Vázquez
@aliciaFPInf

BY NC SA

01

Cluster

El problema que tenemos y resolvemos

Hasta ahora, hemos contratado **ayudantes** (fork o workers) para tareas puntuales o otras cocinas con su cocinero. Pero si el restaurante está lleno de gente haciendo cola en la puerta, un solo Jefe de Cocina no puede ni siquiera tomar las comandas.

El **Cluster** es como abrir 8 restaurantes idénticos (si tienes 8 núcleos) en la misma calle, todos con el mismo nombre y número de teléfono (el mismo Puerto 3000). Hay un Gerente (**Master**) en la puerta que reparte a los clientes: "Tú al restaurante 1, tú al 2...".

Jerarquia

Resumen de la jerarquía de cómo **Node.js** escala:

1. Nivel Micro: worker_threads (Hilos)

Uso: Una tarea de cálculo dentro de una sola petición.

Relación: Es como si el cocinero usara sus dos manos para batir más rápido. Ayudante

2. Nivel Medio: child_process (fork, spawn, exec)

Uso: Delegar una tarea a un proceso externo o script JS.

Relación: Es como mandar a un pinche a la bodega a por vino mientras tú sigues cocinando. Contratar un nuevo cocinero.

3. Nivel Macro: cluster (Procesos gemelos)

Uso: Clonar toda la API para aprovechar todos los núcleos de la CPU.

Relación: Cada "Worker" del cluster es un proceso independiente que a su vez podría usar sus propios forks o worker_threads.

glosario

Clúster: Un grupo de ordenadores conectados (nodos) que trabajan en conjunto como un solo sistema para procesar grandes cantidades de datos o tareas complejas.

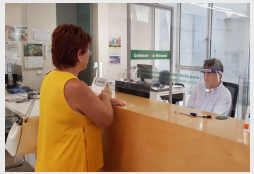
Núcleos (Cores): Son las unidades de procesamiento físicas dentro de una CPU que ejecutan instrucciones.

Significado conjunto: Al diseñar un clúster donde la división de trabajo se ajusta al número de núcleos disponibles en cada nodo (por ejemplo, usar 16 núcleos en un procesador AMD R9 3950X distribuido en 2 clusters internos de 8 núcleos o CCDs), se busca eliminar cuellos de botella.

Objetivo:

- **Procesamiento paralelo:** Dividir un trabajo grande en partes más pequeñas (fragmentos) y procesar cada fragmento en un núcleo/nodo diferente simultáneamente.
- **Alta disponibilidad:** Si un nodo falla, otros pueden asumir la carga.
- **Eficiencia:** Optimizar el rendimiento y el consumo energético aprovechando todos los recursos de hardware disponibles.

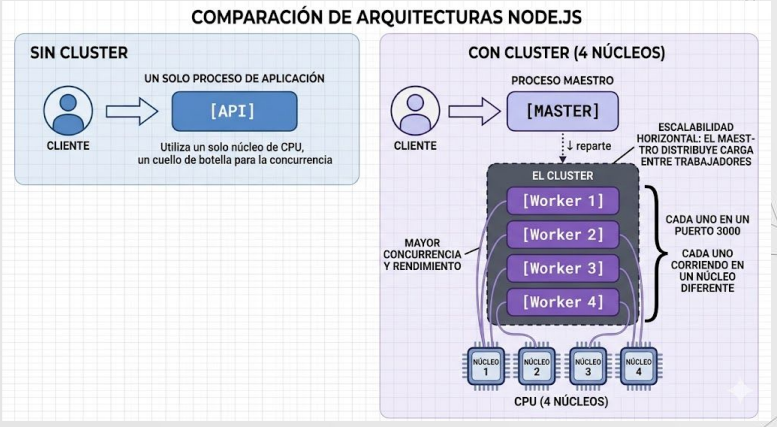
Cluster



Imagina una oficina de atención al cliente con una sola ventanilla. Aunque tengas 8 empleados disponibles, solo uno atiende.

Con cluster abres todas las ventanillas, puedes atender a varios clientes a la vez, tantos como ventanillas tengas. este modelo requiere la implementación de un Master (Dispensador de tickets) para ir repartiendo de manera balanceada los clientes en la diferentes ventanillas.

En el caso de nuestra API deberemos crear un proceso **Master** que no atiende peticiones, solo las **gestiona**. Los **workers** (ventanillas) son copias idénticas de nuestra API que sí atienden.



cluster

Sabemos que **Node** por diseño usa **un solo hilo**. El módulo **cluster** permite arrancar varios procesos Node (uno por núcleo de CPU) que comparten el mismo puerto y se reparten el trabajo.

health check

¿Qué es?

Un **endpoint especial** que responde si la API está viva y funcionando. Lo usan:

- Herramientas de monitorización
- Balanceadores de carga (si un worker no responde, dejan de enviarle tráfico)
- Docker/Kubernetes (para saber si reiniciar el contenedor)
- Nosotros mismos para saber que todo va bien

Un **health check** también comprueba que los servicios críticos también funcionan.

Si se recibe la señal de parada de algún endpoint, el servidor:

- Deja de aceptar conexiones nuevas
- Termina las peticiones que están en curso
- Solo entonces se cierra

server.js

GET http://localhost:3001/health

Query | Headers 2 | Auth | Body | Tests | Pre Run

Query Parameters

parameter	value

Status: 200 OK Size: 49 Bytes Time: 19 ms


```
{
  "status": "ok",
  "pid": 19851,
  "uptime": 46.842350375
}
```


```
aliciavazquezlopez@Ordenador-portatil-de-Alicia AEA03 % npm run dev

> servidor-con-npm@1.0.0 dev
> nodemon server.js

[nodemon] 3.1.9
[nodemon] to restart at any time, enter `rs`
[nodemon] watching path(s): *.*
[nodemon] watching extensions: js,mjs,cjs,json
[nodemon] starting `node server.js`
Servidor arrancado con 8 núcleos disponibles.
Master 19850 arrancado — creando 8 workers...
Servidor arrancado con 8 núcleos disponibles.
Worker 19851 escuchando en http://localhost:3001/
Servidor arrancado con 8 núcleos disponibles.
Servidor arrancado con 8 núcleos disponibles.
Worker 19857 escuchando en http://localhost:3001/
Servidor arrancado con 8 núcleos disponibles.
Servidor arrancado con 8 núcleos disponibles.
Worker 19856 escuchando en http://localhost:3001/
Servidor arrancado con 8 núcleos disponibles.
Worker 19855 escuchando en http://localhost:3001/
Worker 19853 escuchando en http://localhost:3001/
Worker 19854 escuchando en http://localhost:3001/
Servidor arrancado con 8 núcleos disponibles.
Worker 19852 escuchando en http://localhost:3001/
Servidor arrancado con 8 núcleos disponibles.
Worker 19858 escuchando en http://localhost:3001/
[Worker 19851] GET /health
[Worker 19856] GET /health
[Worker 19857] GET /health
[Worker 19853] GET /health
[Worker 19855] GET /health
[Worker 19854] GET /health
[Worker 19852] GET /health
[Worker 19858] GET /health
[Worker 19851] GET /health
```

a tener en cuenta...

Una limitación a tener en cuenta es que cada worker tiene su **propia memoria**. Si guardamos datos en un array en memoria, cada worker tiene su copia:

- `Worker 1: alumnos = [Aina, Marc]  ← POST aquí añade a este array`
- `Worker 2: alumnos = [Aina, Marc]  ← GET aquí puede no ver el nuevo`
- `Worker 3: alumnos = [Aina, Marc]`

Por eso en producción real se usa una **base de datos**, es el almacén compartido entre todos los workers.

Ejercicio guiado: Prueba de carga con autocannon

Autocannon es una herramienta de **benchmarking** (consiste en hacer un "examen de estrés" a una aplicación para conocer sus límites) HTTP de alto rendimiento para Node.js. Se utiliza para hacer pruebas de carga y medir el rendimiento de servidores web y APIs.

Características principales

- Envía miles de peticiones HTTP concurrentes por segundo
- Mide métricas como latencia, throughput (peticiones/segundo) y errores
- Se puede usar como CLI o como módulo de Node.js